Co-Pilot AI Scientist v3: Human-Guided Hypothesis Evolution and Programmatic Search for Collaborative Automated Research

Author: He Shi, School of Computing, National University of Singapore

Abstract

Autonomous research agents have begun to connect idea generation, experiment execution, benchmark evaluation, and paper writing. Yet fully autonomous pipelines still struggle at the creative and strategic decisions where human scientists contribute taste, field knowledge, and responsibility for claims. We propose Co-Pilot AI Scientist v3, a human-in-the-loop architecture that upgrades AI Scientist-v2 by adding structured intervention nodes at hypothesis formation, branch selection, evaluator design, and final claim auditing. The system combines three complementary ideas: AI Co-Scientist-style hypothesis generation and debate, AI Scientist-v2-style experiment automation and manuscript generation, and AlphaEvolve-style programmatic search for machine-gradeable subproblems. We define a reproducible evaluation protocol that compares autonomous, partially gated, and fully collaborative variants on small automated-research tasks. The central hypothesis is that limited human attention, placed at high-leverage nodes, can improve novelty, rigor, and evidence alignment without giving up the scalability of agentic search.

1. Introduction

The next useful version of automated research is unlikely to be a scientist-free paper factory. Scientific work is not only a sequence of executable steps; it is also a process of choosing valuable questions, recognizing weak evidence, reframing failures, and deciding which claims deserve to be made. AI Scientist-v2 demonstrates that agentic tree search can turn hypotheses into experiments and draft papers. AI Co-Scientist demonstrates that multi-agent systems can generate, debate, and refine scientific hypotheses. AlphaEvolve demonstrates that LLM-guided program evolution can produce strong discoveries when automatic evaluators exist. These systems point toward a combined architecture: an automated research co-pilot that lets machines search broadly while allowing humans to intervene where judgment matters most.

This paper proposes Co-Pilot AI Scientist v3. The goal is not to slow an autonomous pipeline with arbitrary approvals, but to identify specific intervention nodes where human input changes the trajectory of research. The proposed nodes are: the creative hypothesis node, the benchmark/evaluator node, the tree-search branch node, the programmatic-search escalation node, and the final claim-audit node.

2. Related Work

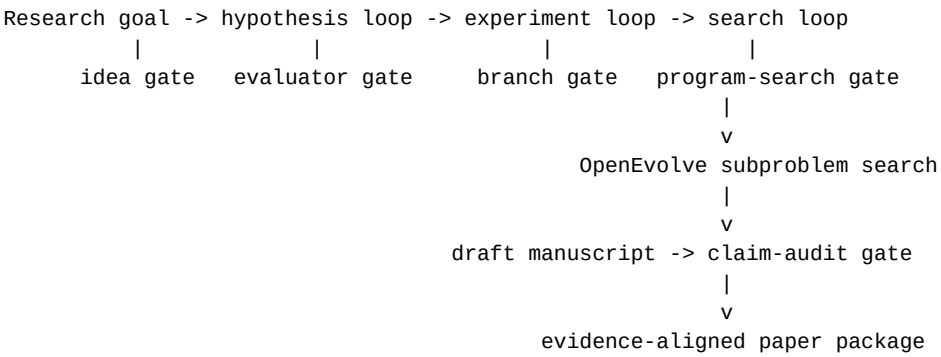
AI Co-Scientist frames scientific discovery as hypothesis generation, discussion, and evolution. Its strength is upstream research imagination and evidence organization. AI Scientist-v2 focuses on turning candidate ideas into runnable experiments and papers through agentic tree search. AlphaEvolve focuses on code evolution driven by automatic evaluation. FunSearch provides an earlier example of LLM-guided program search for mathematical discovery. Coscientist shows how LLM agents can connect to chemistry tools and laboratory automation.

Our proposal treats these systems as complementary layers rather than competing end-to-end solutions.

3. Method

Co-Pilot AI Scientist v3 contains four loops.

Figure 1 gives the operational data flow. A research goal enters the hypothesis loop, passes through experiment construction and AI Scientist-v2-style search, optionally delegates machine-gradeable subproblems to OpenEvolve-based program search, and finally reaches manuscript generation and claim auditing. Each human gate consumes a structured frontier and produces a logged decision artifact.



First, a hypothesis loop generates candidate research directions, critiques them, links them to evidence, and asks the human scientist to select or rewrite the most promising directions.

Second, an experiment loop converts selected hypotheses into benchmark tasks, baselines, ablations, and executable scripts. The human scientist can approve the evaluator before expensive runs begin.

Third, a search loop runs AI Scientist-v2-style tree search over experimental branches. At predefined checkpoints, the system summarizes the branch frontier and asks the human scientist to allocate further budget.

Fourth, a programmatic optimization loop sends machine-gradeable subproblems to an AlphaEvolve-like engine. Since AlphaEvolve itself is not open sourced, our reproducible implementation uses OpenEvolve as the practical substrate for this module. OpenEvolve provides an evolutionary coding loop with custom evaluators, OpenAI-compatible model routing, MAP-Elites quality-diversity search, island-based populations, and reproducible seeds. The module evolves code, stores candidate programs, and returns verified improvements to the main research pipeline.

Each human intervention is recorded as structured data: decision type, options, free-text rationale, affected artifacts, and downstream consequences. This makes human participation part of the reproducibility record rather than an invisible side channel.

The current artifact package contains a retrospective full-gate trajectory covering all proposed gate types. It links an idea gate selecting the narrow human-gated tree-search contribution, an evaluator gate rejecting a fairness metric-gaming branch, a live branch gate selecting among two Causality drafts, a program-search gate escalating to OpenEvolve, and a claim gate demoting unsupported superiority claims to future work. This trajectory demonstrates the log format and evidence chain, but it is not yet a single online end-to-end run.

We also provide a rerunnable full-gate trace script. Given the archived experiment summaries, the script recomputes a continuous sequence of idea, evaluator, branch, program-search, and claim gates and writes both JSON and Markdown artifacts. This verifies that the gate policy can be executed over the current evidence package. It is deliberately labeled as an executable artifact replay rather than a fresh online training run.

4. Benchmark Selection and Evaluation Plan

We evaluate six variants: autonomous baseline, idea gate only, branch gate only, evaluator gate only, claim gate only, and full co-pilot v3. Candidate tasks should be selected by claim type rather than by a single favorite benchmark. For AI Scientist-v2-style experiment search, FML-bench is a useful near-term benchmark because it is already runnable in our Ubuntu environment and exposes branch-level logs. For machine-gradeable algorithmic subproblems, we use OpenEvolve-controlled tasks such as function minimization and knapsack heuristic search. For broader evidence beyond FML-bench, we include an initial MLAgentBench vectorization probe for end-to-end ML experimentation and track setup probes for additional MLAgentBench and ScienceAgentBench tasks. The current evidence still does not include a second scored official non-FML benchmark: the MLAgentBench CIFAR10/debug run was blocked by slow dataset download, and ScienceAgentBench metadata/artifacts were not reachable from the Ubuntu host. Higher-cost stretch benchmarks include MLE-bench Lite for Kaggle-style ML engineering, PaperBench for paper-to-code replication and hierarchical rubric grading, and AIRS-Bench for full ML research lifecycle evaluation.

Metrics include task score, paper quality, claim support, search efficiency, human attention cost, and hypothesis diversity. For the programmatic-search module, the key ablation is OpenEvolve versus repeated direct LLM edits under the same evaluator and iteration budget. FML-bench should therefore be read as one current evidence source, not as the complete benchmark definition for the project.

To make this principle operational, we maintain a benchmark-to-claim matrix. FML-bench Causality supports the branch-gate feasibility claim but does not yet show general human-gate superiority. FML-bench Fairness supports the need for evaluator gates because executable repairs and degenerate predictors reveal metric-gaming risk. OpenEvolve function minimization, knapsack, MLAgentBench vectorization, and sklearn diabetes probes test the program-search escalation policy under different subproblem types. ScienceAgentBench, MLE-bench Lite, PaperBench, and AIRS-Bench remain expansion targets rather than current scored claims. Future runs should therefore be chosen by the weakest unsupported claim, not by convenience.

4.1 Preliminary Programmatic-Search Smoke Test

As an initial feasibility check, we ran OpenEvolve 0.2.27 on the SSH-controlled Ubuntu host using DeepSeek through an OpenAI-compatible API. The task was a small function-minimization evaluator with a random-search baseline. In one OpenEvolve iteration, the system improved the evaluator score from 0.0345 to 0.0378 and saved a simulated-annealing-style evolved program. This result is only a smoke test: it verifies the remote OpenEvolve path, evaluator loading, model routing, checkpointing, and artifact capture. It does not yet establish the paper-level claim that human-gated research improves final manuscript quality.

We also ran a direct one-shot LLM-edit baseline with the same DeepSeek model, initial program, and evaluator. The direct edit reached a score of 0.038021, slightly above both the one-iteration OpenEvolve score of 0.037816 and a five-iteration OpenEvolve score of 0.038007. This boundary result supports a central design choice of Co-Pilot AI Scientist v3: programmatic search should be controlled by an explicit escalation gate. For small budgets or simple local rewrites, direct editing may be sufficient; for richer search spaces and larger budgets, OpenEvolve-style population search can become the appropriate tool.

To test that latter case, we added a richer 0/1 knapsack heuristic task. The initial value/weight greedy program reached an average optimality ratio of 0.991519 across 18 deterministic instances. A direct LLM rewrite improved this to 0.995270. A five-iteration OpenEvolve run improved it further to 0.999439, with no invalid instances. The evolved program introduced local improvement via one-item and two-item removals followed by greedy refill. This provides a positive example for the escalation gate: OpenEvolve-style search can be useful when the subproblem has a richer combinatorial structure and an automatic evaluator.

4.2 Retrospective AI Scientist-v2 Branch-Gate Replay

We also replayed two existing AI Scientist-v2 FML-bench smoke runs. In the Causality_causalml run, the first draft achieved the best validation MAE observed in the four-step run (0.598943 versus a baseline of 1.296259), while the second draft and later refinements failed to improve it. A branch gate after two drafts would have kept the first branch and avoided at least one low-value follow-up. In the Fairness_fairlearn run, the first draft worsened the baseline fairness metric (0.316467 versus baseline 0.186632), the second attempt failed with a Fairlearn compatibility issue, and later attempts were either worse or buggy. A human branch gate would have paused the branch and routed the system to an evaluator or algorithm reset. This replay evidence is not a substitute for an online human-gated benchmark, but it shows that the logged AI Scientist-v2 trajectory contains actionable checkpoints for human co-pilot intervention.

4.3 Live AI Scientist-v2 Branch-Gate Probe

To move beyond retrospective replay, we ran a new live FML-bench probe on the Ubuntu host. We configured AI Scientist-v2 on Causality_causalml with a two-step budget, two draft ideas, two simulated parallel workers, and a temporary stage-budget schedule that assigns all budget to the first stage. This produced a real two-branch frontier before a branch gate. The first draft obtained validation MAE 1.149610, while the second draft obtained validation MAE 0.621262; lower is better. The structured branch gate therefore selected the second draft and pruned the first. The benchmark's automatic final test on the selected validation branch reported test MAE 0.640451.

This live probe demonstrates that the proposed branch gate can be inserted at a real AI Scientist-v2 decision frontier and can make a clear budget-allocation decision from logged metrics and code snapshots. It does not yet prove that human gating outperforms autonomous AI Scientist-v2, but it gives a concrete frontier on which such a comparison can be built.

We then implemented a minimal selected-branch continuation runner. The runner temporarily applies the human-selected code snapshot to the official FML-bench task template, launches a short AI Scientist-v2 continuation run, and restores the template afterward. Starting from the selected second draft, a two-step continuation reached validation MAE 0.401240 and test MAE 0.402170. This is better than the live two-draft gate run's test MAE of 0.640451 and also better than the earlier four-step autonomous smoke run's test MAE of 0.617719. The comparison is still preliminary because continuation is implemented by snapshot seeding rather than by preserving the original in-memory tree object, and it uses only a single small task and seed. Nevertheless, it demonstrates an executable path from branch gate, to selected snapshot, to additional AI Scientist-v2 search budget.

Following the paper-quality review, we added closer matched-budget autonomous baselines. The first matched run used the same Causality_causalml task, the same DeepSeek model, two initial ideas, two parallel branches, and four total AI Scientist-v2 steps, but no human branch selection. It reached validation MAE 0.389451 and test MAE 0.421474. In this pair, the human-gated path remained slightly better on held-out test MAE (0.402170 versus 0.421474), while the autonomous baseline was slightly better on validation MAE.

We then ran a second matched Causality replicate. The human-gated two-draft frontier selected a branch with validation MAE 0.605881 and test MAE 0.646224. The snapshot-seeded continuation did not improve the held-out test score, ending with validation MAE 0.627837 and test MAE 0.646224. The matched autonomous four-step baseline reached validation MAE 0.537972 and test MAE 0.595685. In this second pair, the autonomous path was better on both validation and held-out test metrics.

Across the two matched pairs, one pair favors human-gated continuation and one favors the autonomous baseline on held-out test MAE. Mean human-gated test MAE is 0.524197, while mean autonomous test MAE is 0.508579. Since lower is better, the two-pair mean slightly favors the autonomous baseline. The evidence therefore removes one compute-budget confound but remains mixed; it supports feasibility of branch-gate insertion and continuation, not general human-gate superiority.

4.4 Non-FML Benchmark and Program-Search Probe

Following the benchmark-selection principle above, we also used MLAgentBench as a non-FML evaluation source. We cloned MLAgentBench on the Ubuntu host and first ran its lightweight vectorization task with the built-in Agent baseline, which simply executes the starter train.py and submits the result. The official baseline completed with final score 3.172504 seconds and total benchmark time 3.365531 seconds, with no reported error flags.

We then wrapped the same task in a stricter local evaluator: before accepting a runtime, the evaluator compares the candidate Conv2DLayer.forward output against a nested-loop reference on a deterministic small input. Under this controlled evaluator, the starter program had median runtime 3.261186 seconds. A direct DeepSeek deepseek-chat rewrite produced plausible vectorized code but failed the correctness gate due to a numerical mismatch. In contrast, a three-iteration OpenEvolve-style run using the same DeepSeek model found a correct candidate at iteration 1 with median runtime 0.051882 seconds, a 62.86x speedup over the controlled starter program. This result supports the narrower claim that the program-search-escalation node can be valuable on machine-gradeable subproblems beyond FML-bench. It does not yet prove that the full co-pilot architecture improves whole-paper quality.

To test robustness, we repeated the same three-iteration OpenEvolve setup with additional random seeds. Across seeds 0, 1, 2, 3, 4, 7, 42, and 123, all eight runs retained a correct best program and improved over the controlled starter. The median best runtime was 0.024581 seconds, corresponding to a median speedup of about 132.67x over the starter runtime. Six of eight seeds found a sub-0.1-second program. The result is still not deterministic under tiny budgets: seed 7 only achieved a weak 1.09x speedup and seed 1 achieved a 15.57x speedup. This strengthens the evidence that the program-search module can find useful code transformations, while preserving the important caveat that search quality varies substantially by seed and budget.

To avoid making the non-FML evidence only about low-level runtime optimization, we added a second controlled probe using the built-in sklearn diabetes tabular regression dataset. This probe does not require Kaggle credentials and should not be reported as an official MLAgentBench score. It starts from a deliberately rudimentary mean predictor with mean RMSE 78.572189 across five deterministic splits. A direct DeepSeek rewrite recovered a standard Ridge-style baseline with mean RMSE 55.895460. Three 3-iteration OpenEvolve seeds also improved strongly over the mean predictor, with best RMSEs 55.895460, 55.946535, and 55.895460. The median OpenEvolve RMSE was 55.895460, matching the direct rewrite. This result broadens the benchmark coverage to a tabular modeling subproblem, but it also gives an important boundary condition: when the improvement is a standard small modeling change, direct editing can be as effective as program search. Thus the program-search gate should be selective, not automatic.

We also attempted two benchmark-expansion probes. First, we tried to add a second official MLAgentBench debug task, which maps to CIFAR10. The setup probe repaired a missing torchvision dependency by installing the matching CPU wheel for the local torch version, but the run stopped during dataset preparation because the 170 MB CIFAR10 archive was downloading too slowly for the interactive budget. Second, we probed ScienceAgentBench. The repository was present on the Ubuntu host, and its README points to the April 2026 verified split and benchmark_verified.zip, but the local benchmark directory did not contain the verified artifacts and the HuggingFace metadata request failed with [Errno 101] Network is unreachable. We therefore report both as setup artifacts and not as benchmark scores.

4.5 Claim Audit

We ran a claim-evidence audit after the pilot experiments. A Monica-routed gpt-4o-mini review classified the core architecture contribution as reasonable but warned that the empirical evidence does not yet support broad claims that human gates improve paper quality or that the full co-pilot system outperforms autonomous AI Scientist-v2. We therefore treat those as hypotheses for the evaluation protocol rather than as conclusions. The audit supports only the narrower empirical claims reported above: OpenEvolve-style search can help on some machine-gradeable subproblems but is seed-sensitive under tiny budgets, direct editing can match OpenEvolve on a simple tabular modeling probe, branch-gate insertion is feasible in AI Scientist-v2-style logs, evaluator gates must reject non-executable and metric-gaming branches before continuation, and the first two matched Causality pairs give mixed evidence rather than a reliable human-gating advantage. A first attempt to extend online branch gating to Fairness_fairlearn produced two validation failures rather than a score, so we archive it as failure-mode evidence rather than matched-budget performance evidence. A follow-up evaluator-gate repair probe made the lesson sharper: an API-repaired fairness candidate was executable but worse on demographic parity difference (test 0.317603 versus the baseline 0.173030), while a degenerate all-negative predictor achieved a perfect target metric of 0.000000 but collapsed balanced accuracy to 0.500000. For fairness tasks, the gate therefore needs both an execution check and a utility floor before accepting a branch.

We also ran a paper-quality review through two Monica-routed reviewer models. gpt-4o-mini gave a weak-accept recommendation with scores of 4/5 for novelty and reproducibility but 3/5 for rigor and evidence. claude-3-7-sonnet-latest was stricter and gave a reject recommendation for a strong ML/NLP systems venue, mainly because the current system is evaluated through isolated module probes rather than a matched-budget end-to-end comparison. Both reviewers converged on the same required next step: run autonomous AI Scientist-v2 and human-gated variants under equal budgets across more tasks and seeds, and measure human attention cost.

5. Current Contributions and Unproven Claims

The current contributions are:

1. A modular architecture for collaborative automated research.
2. A formal schema for human intervention nodes in research agents.
3. A retrospective full-gate trajectory showing idea, evaluator, branch, program-search, and claim-audit gates serialized under the shared schema.
4. A rerunnable full-gate trace script that recomputes the gate chain from archived experiment summaries while marking the output as artifact replay.
5. An evaluation protocol for measuring whether and where human attention improves agentic research.
6. Initial remote OpenEvolve and FML-bench probes showing that the

AlphaEvolve-style subproblem module and AI Scientist-v2 branch-gate module can run on the Ubuntu host.

7. Two matched-budget FML-bench Causality comparisons between a human-gated branch continuation and a four-step autonomous AI Scientist-v2 baseline, with mixed outcomes.

8. Non-FML program-search probes for runtime optimization and tabular regression, broadening benchmark coverage beyond FML-bench.

9. A reusable Codex skill for running the workflow.

10. Bilingual paper, usage artifacts, and claim-audit artifacts for reproducibility.

11. A Monica-routed paper-quality review artifact that records external model criticism before the next revision.

The current evidence does not yet prove that human gates improve paper quality or that the full co-pilot system outperforms autonomous AI Scientist-v2. Those remain target claims for the next benchmark stage.

6. Limitations

This proposal does not assume that human involvement always helps. Human gates can introduce bias, slow search, and reduce exploration. Programmatic search can optimize local metrics without improving scientific contribution, and under tiny budgets it may not outperform direct LLM editing. Expert paper ratings are costly and may vary across reviewers. The first version should make narrow claims and report negative results when gates fail to improve outcomes. The current selected-branch continuation evidence is mixed across two matched pairs and is not yet a statistically controlled benchmark. The retrospective full-gate trajectory and executable artifact replay show the schema, decision chain, and reproducible traversal logic, but neither is a single online run with all gates active. Stronger claims require more tasks, more seeds, richer budget schedules, a true online four-loop trajectory, and independent paper-quality review.

The current implementation also lacks one complete end-to-end demonstration in which all four loops operate in a single continuous trajectory. The module probes show feasibility for individual components, but the next systems-paper draft must report at least one full trajectory from hypothesis generation to final claim-audited manuscript.

7. Conclusion

Co-Pilot AI Scientist v3 reframes automated scientific discovery as collaborative search. The system preserves the strengths of autonomous agents while giving human scientists explicit, logged, and experimentally testable points of influence. If future matched benchmarks support the central hypothesis, this architecture could produce better research papers not by removing humans from science, but by using human attention where it has the highest marginal value. The present paper should be read as a reproducible system proposal with pilot evidence, not as a final proof of superiority.